Paged KV-Cache Management and Continuous Batching

SYNTHETIC TEST DOCUMENT. All values in this document were invented to test the OmniResearch software and do not report real experiments.

Abstract

Naive contiguous KV-cache allocation wastes 60% to 80% of reserved KV memory through fragmentation and over-reservation. Paging the KV cache into fixed blocks of 16 tokens reduces waste to under 4%. Combined with continuous (iteration-level) batching, serving throughput increases by 23x relative to static batching on the same A100 40 GB GPU.

1 Background

During autoregressive decoding each token's keys and values are cached. For a 13B model with 40 layers and hidden size 5120, the KV cache needs about 0.8 MB per token in FP16, so a 2048-token sequence needs about 1.6 GB.

2 Paged Attention

The cache is split into blocks; a block table maps logical to physical blocks, as in virtual memory. Blocks can be shared between sequences for parallel sampling, with copy-on-write.

3 Continuous Batching

Static batching waits for every sequence in a batch to finish. Continuous batching schedules at every decoding iteration, admitting new requests as soon as others complete, which keeps the GPU saturated.

4 Results

Figure 1 shows throughput in requests per second for three serving configurations on an A100 40 GB.

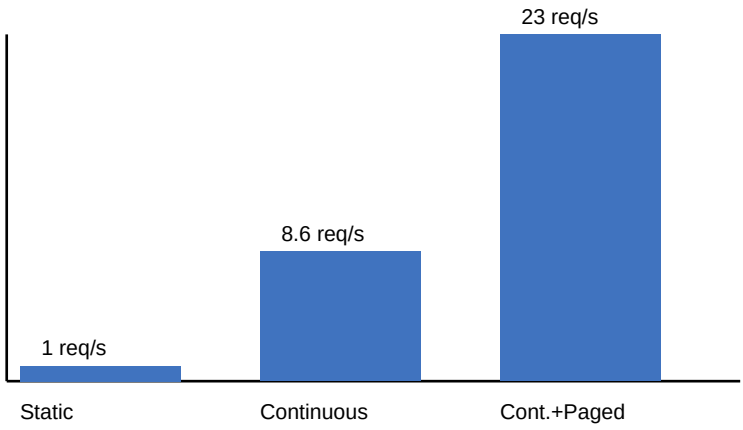

Figure 1: Serving throughput (requests/s) of static batching, continuous batching, and continuous batching with paged KV cache.

Configuration	KV memory waste	Throughput (req/s)	Max batch
Static batching	71%	1.0	8
Continuous batching	71%	8.6	8
Continuous + paged KV	3.8%	23.0	96

Table 2: KV-cache waste and throughput by configuration.

5 Limitations

Paging adds an indirection in the attention kernel, costing roughly 3% to 5% kernel latency. Results are for a single GPU.